

Задание 1. Архитектурная диагностика и целевое видение

1. Анализ текущей архитектуры

1.1 Диаграммы текущего состояния системы

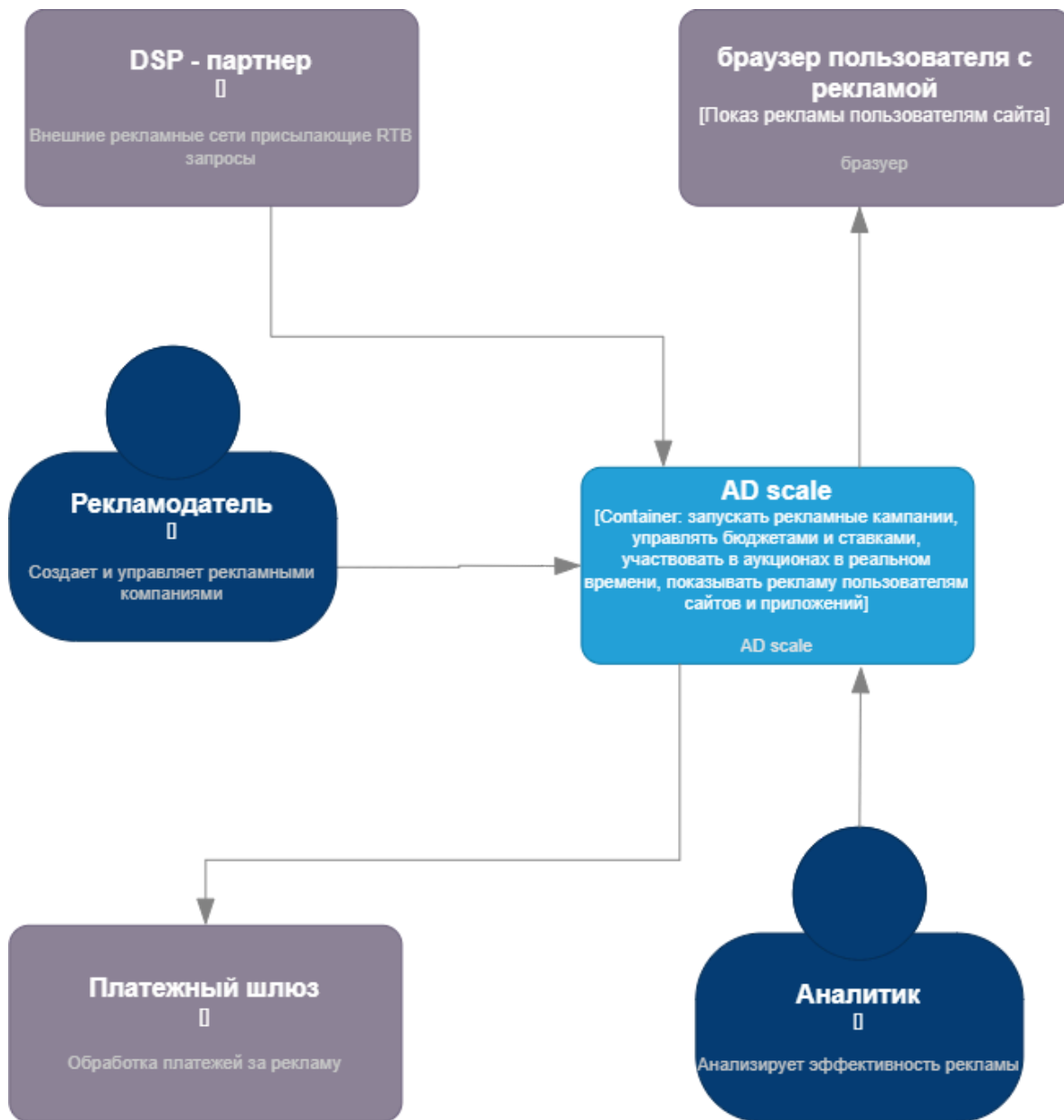


Диаграмма контекста в нотации C4

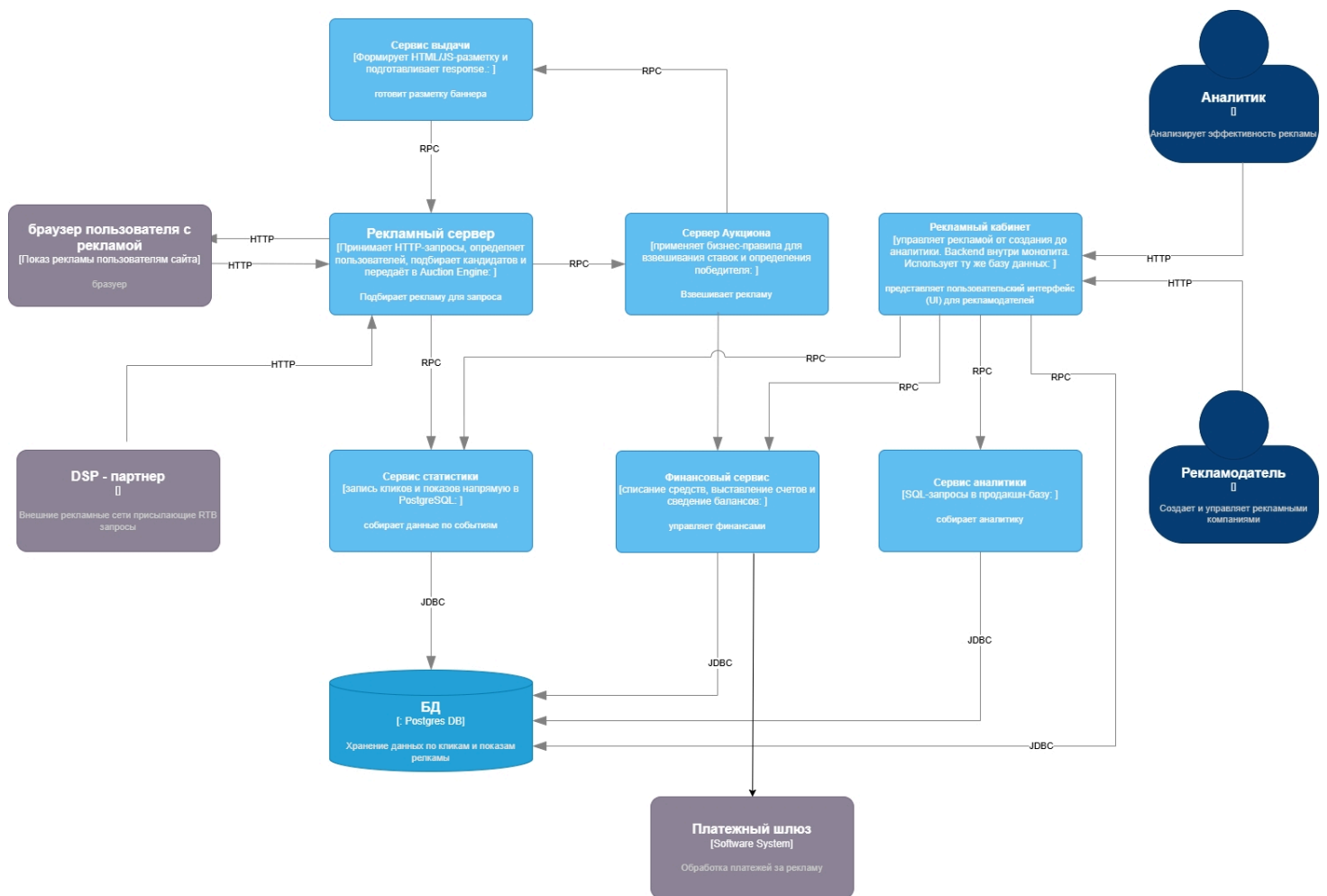


Диаграмма контейнеров в нотации C4

1.2 Организационные ограничения

- Ограниченный персонал. Выделенных SRE-инженера, технического писателя и бизнес-аналитика в компании нет;
- Архитектура платформы не задокументирована – сложность поиска неисправностей;
- Отсутствие возможности переписать систему полностью;
- Требуется постепенная миграция на целевую архитектуру

1.3 Текущая нагрузка

- До 15 000 RPS в пике на аукцион;
- P95 времени отклика составляет 110–120 ms при требовании не более 100 ms.
- Периодические деградации в часы пик.

- Отсутствие запаса по производительности.

1.4 Узкие места

- Система AdScale работает на пределе своих возможностей по производительности
- Ожидается существенный прирост нагрузки в процессе интеграции с DSP-платформой;
- Единая БД для всех модулей как для высокочастотных запросов в реальном времени (RTB) так для аналитики со сложными SQL запросами;
- Отсутствие резервирования БД и серверов приложений для продуктивной среды;
- Синхронные коммуникации: проблемы производительности одного компонента влияют на связанные модули системы;

1.5 Архитектурные проблемы и ограничения

- Монолитная (модульная) архитектура – отсутствие возможности горизонтального масштабирования;
- Возможности вертикального масштабирования ограничены монолитной архитектурой приложения;
- Использование единой БД Postgres (сервер БД масштабируется только вертикально) также ограничивает производительность;
- Отсутствие кеша для чтения из БД Postgres что снижает производительность по чтению из БД;
- Скрытые зависимости — общая БД создаёт сильную связанность между модулями;
- Не используются очереди – вся коммуникация между модулями синхронная.
- Ручной деплой, отсутствие инструментов конвейера разработки и тестирования и соотв-но контейнеризация

1.6 Результаты анализа текущей архитектуры

- Архитектура — модульный монолит.
- База данных — одна PostgreSQL, все модули работают с общей схемой.
- Отсутствие кеширования, поэтому каждый поиск билетов идёт напрямую в базу данных.
- Доступно только ограниченное вертикальное масштабируемое.
- Нет очередей, а все операции синхронные, так что при пиковой нагрузке потоки блокируются.
- До 15 000 RPS в пике на аукцион;
- Строгие требования по времени отклика: не более 100 ms для аукциона;
- Высокая нагрузка на запись (события кликов и показов);
- Постоянный поток событий (event stream).

Проблема	Причина
Рост latency	Прямые обращения к БД
Деградация при нагрузке	Синхронная модель
Ограничение масштабирования	Монолит модульный
Конкуренция за ресурсы	Общая БД

1.7 Ключевые требования для выбора целевой архитектуры и миграции

- Увеличение нагрузки на 20 процентов после миграции
- Увеличение нагрузки в 2 раза через год
- Время отклика сервиса аукционов до не более 80 ms для аукциона
- Надежность изоляция сбоев, быстрое восстановление – 99,9
- Независимость работы компонентов/сервисов системы
- Гибкого масштабирование системы
- Невозможно переписать систему полностью, требуется постепенная миграция

Архитектура в виде модульного монолита не решает основной проблемы — конкуренцию за дисковую подсистему единственной инсталляции PostgreSQL. Очереди сообщений, кеширование, репликация/шардирование необходимы.

Вывод – ключевые требования могут быть выполнены при переходе на микросервисную архитектуру.

Требуется постепенная миграция на целевую микросервисную архитектуру

2. Определение архитектурных драйверов

Приоритет	Категория	Драйвер	Значения
Высокий	Функциональное требование	Бизнес деятельность	Запуск рекламных компаний
Высокий	Функциональное требование	Бизнес деятельность	Управление рекламным бюджетом
Высокий	Функциональное требование	Бизнес деятельность	Участие в аукционах в реальном времени
Высокий	Функциональное требование	Бизнес деятельность	Показ рекламу пользователям сайтов и приложений
Средний	Функциональное требование	Бизнес деятельность	Сбор аналитики
Средний	Функциональное требование	Бизнес деятельность	Формирование отчётности
Средний	Функциональное требование	Бизнес деятельность	Мульти платформенная интеграция с с DSP-площадкой
Высокий	Атрибут качества	Масштабируемость	До 18 000 RPS в пике через 3 месяца
Высокий	Атрибут качества	Масштабируемость	До 50 000 RPS в пике через 3 месяца
Средний	Атрибут качества	Масштабируемость	Горизонтальное масштабирование
Средний	Атрибут качества	Масштабируемость	Мульти региональное развёртывание
Средний	Атрибут качества	Производительность	Не более 80 ms для аукциона
Средний	Атрибут качества	Надежность	Доступность 99,9 %
Средний	Атрибут качества	Надежность	Изоляция сбоев и быстрое восстановление после сбоев
Средний	Атрибут качества	Надежность	Независимая работа компонентов / сервисов
Высокий	Техническое ограничение	Команда	Невозможно переписать систему полностью, требуется постепенная миграция
Низкий	Техническое ограничение	Стек	C++ / Go / Python / Node
Высокий	Бизнес-ограничение	Сроки	3 месяца для промежуточного решения
Высокий	Бизнес-ограничение	Сроки	1 целевого решения год
Высокий	Бизнес-ограничение	Бюджет	Ограничен
Высокий	Бизнес-ограничение	Непрерывность	Отсутствие ответа в заданные ограничения ведет к проигрышу аукциона и потере прибыли

Низкий	Бизнес-ограничение	Команда	35 человек: backend-, frontend- и DevOps-разработчики, тестировщики, дизайнеры, сотрудники службы поддержки и менеджмент.
--------	--------------------	---------	---

3. Выбор архитектурного стиля

3.1 Выбор метода миграции

Сравнительный анализ альтернатив:

- Модульный монолит: Позволяет упорядочить кодовую базу внутри единого процесса, однако не решает фундаментальных инфраструктурных проблем текущей системы — отсутствия очередей, высокой конкуренции за I/O единственной инсталляции PostgreSQL и синхронных межмодульных вызовов. Данный подход недостаточен для обеспечения целевого P95 < 100 ms при росте нагрузки.
- Переписать все с нуля («Big Bang»): Неприменима ввиду критического риска нарушения непрерывности бизнеса, отсутствия запаса производительности и высоких требований к компетенциям команды в области распределенных систем.
- Паттерн «Strangler Fig» (Фиговое дерево-душитель): Является наиболее сбалансированным решением для долгосрочной перспективы, так как обеспечивает постепенный вывод легаси-компонентов из эксплуатации без остановки обработки трафика.

Предлагается осуществить поэтапную стратегию, выбрав за основу Strangler Fig Pattern как наиболее универсальный подход в постепенной и безопасной миграции на микросервисы.

Strangler Fig Pattern - суть паттерна:

- Пре реquisite: существующая монолитная система разделена на модули (готово);
- Создается реверсивный прокси или шлюз для доступа к модулям и перспективным микросервисам
- Выбранный модуль (например, модуль аналитики) выносится в отдельный микросервис аналитики;
- Реверсивный-прокси (или шлюз) направляет запросы следующим образом:
- Существующий код в монолите обрабатывает все функции, кроме вынесенной,
 - Новые запросы для модуля Статистики (например) перенаправляются в микросервис аналитики;
 - Далее постепенно остальные модули выносятся в аналогичные микросервисы.

Основные достоинства данного подхода:

- Простой и понятный подход без дополнительной разработки.
- Минимизируется риск: существующий код монолита остается в работе;

- Возможность быстрого отката существующую архитектуру монолит
- Позволяет разделить процесс перевода на отдельные шаги и решать задачу поэтапно;

Тем не менее, внедрение паттерна Strangler Fig без подготовки инфраструктуры и решения первоочередных проблем с производительностью не приведет к требуемым результатам, а только добавит сетевых задержек и новых точек отказа.

3.2 Стратегия миграции

1 этап. Повышение производительности, создание инфраструктуры и повышение производительности (18 000 RPS и соблюдение SLA (< 80 ms latency)) для дальнейшего внедрения целевого решения:

- **Внедрение асинхронной модели записи** в существующей архитектуре. Сервер сбора статистики отключаются от прямого доступа к БД. Создается брокер сообщений (кластер Apache Kafka). Запись событий осуществляется через запись в топик со временем отклика ~1–2 мс. Далее запись в БД производится путем чтения из топика в фоновом режиме.
- **Разделение транзакционных и аналитических потоков.** Разворачивается Read Replica PostgreSQL. SELECT-запросы сервиса аналитики и рекламного кабинета полностью перенаправляются на реплику.
- **Создание единой точки входа.** Создается реверсивный прокси перед внешними системами и пользователями, которая в последующем будет выполнять роль API Gateway (Envoy, Traefik, NGINX+, HAProxy) для управления маршрутизацией, аутентификацией и применения политик Circuit Breaker.

2 этап Постепенный вынос модулей/сервисов в микросервисы согласно выбранному паттерну Strangler Fig

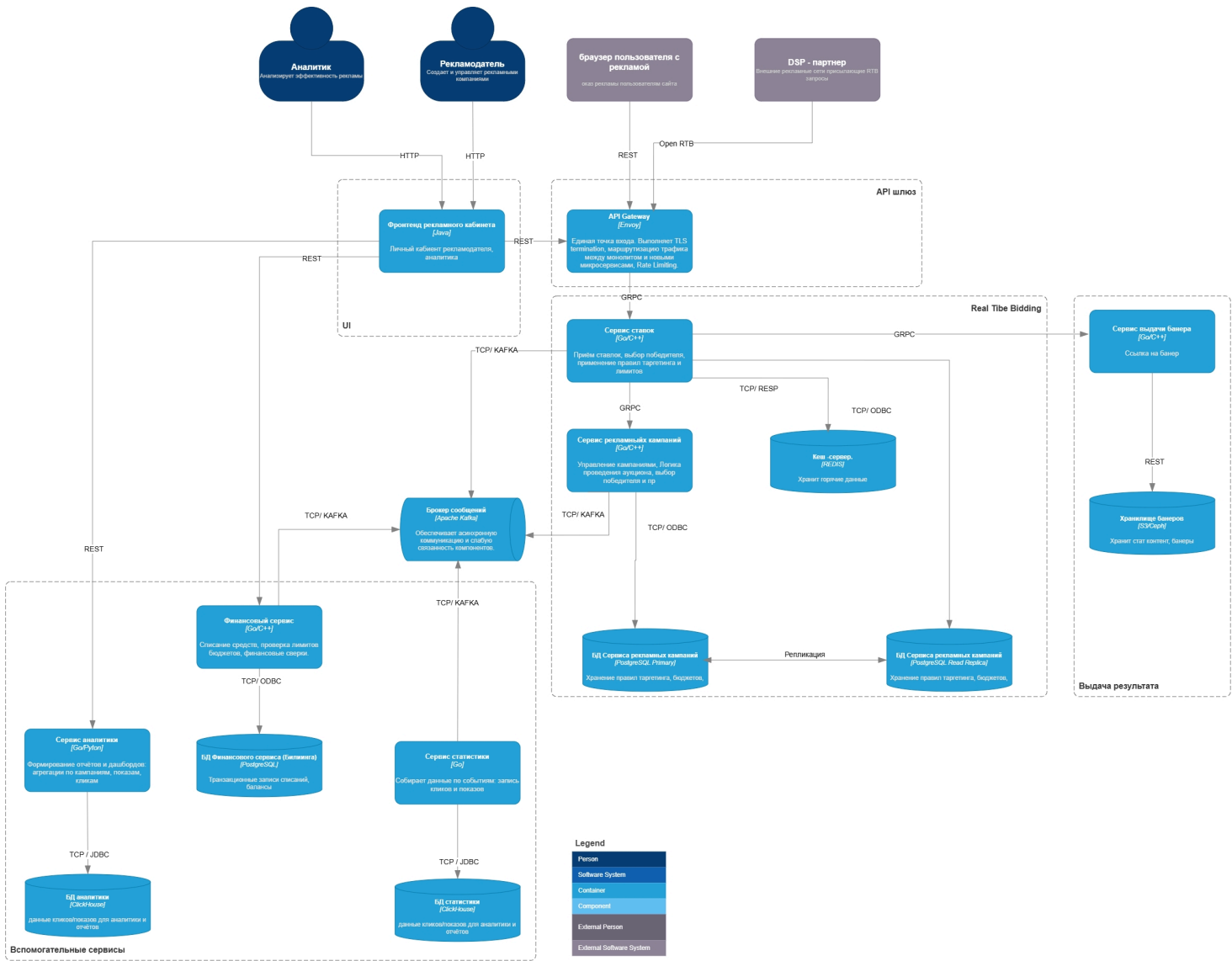
- **Изоляция сервисов статистики и аналитики.** Создается независимые микросервисы статистики и аналитики с собственными БД (или репликой БД). Монолит выступает исключительно в роли источника данных (событий) в Kafka.
- **Повышение времени отклика для чтения из БД.** Внедряется кеширование данных в Redis Cluster для исключения блокировок PostgreSQL БД.

- **Вынос бизнес-логики в отдельный высокопроизводительные микросервисы** (C++/Go), доступ к которому осуществляется по gRPC. Это повышает производительность и снижает сложность системы в целом.
- **Управление данными.** Создание выделенных БД для новых микросервисов. Запрет на прямые SQL-обращения новых сервисов к таблицам старого монолита. Синхронизация данных новых БД с мастер данными в существующей БД через механизм Change Data Capture (Debezium).
- **Изоляция финансовых других операций.** Финансовые расчёты также становятся асинхронным и исключаются из критического пути ответа на ставку. Также выносятся в микросервисы оставшиеся модули/сервисы.

3. Этап Горизонтальное масштабирование и финальная миграция

- **Шардирование** наиболее нагруженных таблиц для БД аукциона по мере роста нагрузки.
- **Повышение количества реплик** наиболее нагруженных сервисов по мере роста нагрузки и балансировка на них через API Gateway
- **Мультирегиональное развертывание.** Архитектура разворачивается в нескольких зонах доступности. Глобальный балансировщик DNS направляет трафик в ближайший регион, снижая сетевую задержку (RTT).

4. Целевая архитектура



5. ADR-001. Стратегия эволюции архитектуры (от монолита к микросервисам).

Статус – предложено

Дата -

5.1 Контекст

Текущая архитектура системы размещения рекламы в формате аукционов в реальном времени AdScale представляет собой модульный (распределенный, гибридный монолит).

Система работает на пределе своих возможностей по производительности обрабатывает до 15 000 RPS с P95 latency 110–120 мс при целевом показателе в 100 мс. Ожидается рост нагрузки на 20% (до 18 000 RPS) к концу текущего квартала и двукратный рост (до 50 000 RPS) в течение года.

Архитектура характеризуется следующими критическими недостатками:

- **Узкое место и точка отказа:** Все компоненты (аукцион, статистика, финансы, аналитика) используют одну инстанцию БД PostgreSQL, что вызывает блокировку транзакций и деградацию времени отклика.
- **Отсутствие возможности масштабирования:** Отсутствие контейнеризации, очередей сообщений и кеширования делает невозможным горизонтальное масштабирование компонентов и всего сервиса .в целом
- **SLA нового партнера:** DSP-площадка требует стабильного ответа не более 80 мс и накладывает штрафы за простой.
- **Бизнес-ограничения:** Бюджет ограничен, срок промежуточного решения — 3 месяца, команда состоит из 35 человек без выделенных SRE. Полная переписывание системы исключено.

Необходимо выбрать стратегию трансформации, которая обеспечит работу под нагрузкой с увеличением на 20% (до 18 000 RPS) через 3 месяца и заложит фундамент для достижения 50 000 RPS через год.

5.2 Рассмотренные варианты

Вариант 1: Модульный монолит

Переход от распределенного монолита к модульному монолиту.

- Плюсы: Минимальные изменения инфраструктуры, сохранение производительности межмодульных вызовов (in-process).
- Минусы: Не решает проблему одновременного доступа к единственной БД. Не позволяет независимо масштабировать сервис статистики или аналитики. Сохраняет риск глобальных сбоев при деплое.

Вариант 2: «Big Bang» переписать все с нуля и переключить на новый сервис

Одновременное разделение монолита на десятки независимых микросервисов.

Не подходит вследствие бизнес-ограничений

Вариант 3: Паттерн Strangler Fig (Выбранный вариант)

Постепенное добавление к существующему коду монолита нового функционала в виде API Gateway и новых сервисов с последующей постепенной заменой устаревших частей.

- Плюсы: Обеспечивает непрерывность бизнеса. Позволяет начать с инфраструктурных изменений (очереди, кеш), дающих мгновенный прирост производительности. Снижает риски каждой итерации.
- Минусы: Временное существование распределенного монолита (гибридной архитектуры). Сложность поддержки двух путей обработки данных во время переходного периода.

Решение

Принять Вариант 3 (Strangler Fig Pattern) как основную стратегию миграции. Миграция разделена на два этапа

Этап 1: Повышение производительности инфраструктуры для возможности подключения DSP партнера. Вынос сервисов некритичных компонентов (аналитика)

Цель: Подключение DSP-партнера с сохранением требуемых SLA.

- **Внедрение асинхронной модели записи** в существующей архитектуре. Сервер сбора статистики отключаются от прямого доступа к БД. Создается брокер сообщений (кластер Apache Kafka). Запись событий осуществляется через запись в топик со временем отклика

~1–2 мс. Далее запись в БД производится путем чтения из топика в фоновом режиме. Прямая синхронная запись в PostgreSQL устраняется.

- **Разделение транзакционных и аналитических потоков.** Разворачивается Read Replica PostgreSQL. SELECT-запросы сервиса аналитики и рекламного кабинета полностью перенаправляются на реплику.
- **Создание единой точки входа.** Создается реверсивный прокси перед внешними системами и пользователями, которая в последующем будет выполнять роль API Gateway (Envoy, Traefik, NGINX+, HAProxy) для управления маршрутизацией, аутентификацией и применения политик Circuit Breaker.

Альтернатива – выделение сервиса аукциона в отдельный микросервис на первом этапе и последующая репликация данного сервиса для возможности горизонтального масштабирования.

Достоинства – повышение производительности критичного сервиса

Недостаток – сложность решения, риск не выдержать срок. Не решает проблему одновременного доступа к единственной БД.

2 этап Постепенный вынос модулей/сервисов в микросервисы согласно выбранному паттерну Strangler Fig

- **Изоляция сервисов статистики и аналитики.** Создаются независимые микросервисы статистики и аналитики с собственными БД (или репликой БД). Монолит выступает исключительно в роли источника данных (событий) в Kafka.
- **Повышение времени отклика для чтения из БД.** Внедряется кеширование наиболее часто читаемых данных в Redis Cluster (например, данные пользователей) для исключения блокировок PostgreSQL БД
- **Вынос бизнес-логики в отдельный высокопроизводительные микросервисы (C++/Go),** доступ к которому осуществляется по gRPC. Это повышает производительность и снижает сложность системы в целом.
- **Управление данными.** Создание выделенных БД для новых микросервисов. Запрет на прямые SQL-обращения новых сервисов к таблицам старого монолита. Синхронизация данных новых БД с мастер данными в существующей БД через механизм Change Data Capture . Рассмотреть выбор БД для сервиса аналитики в отдельном ADR – предпочтительный тип БД для данного сервиса, ClickHouse для оптимизации OLAP-нагрузки.

- **Изоляция финансовых других операций.** Финансовые расчёты также становятся асинхронным и исключаются из критического пути ответа на ставку. Также выносятся в микросервисы оставшиеся модули/сервисы.

3. Этап Горизонтальное масштабирование и финальная миграция

- **Шардирование** наиболее нагруженных таблиц для БД аукциона по мере роста нагрузки.
- **Повышение количества реплик** наиболее нагруженных сервисов по мере роста нагрузки и балансировка на них через API Gateway
- **Мультирегиональное развертывание.** Архитектура разворачивается в нескольких зонах доступности. Глобальный балансировщик DNS направляет трафик в ближайший регион, снижая сетевую задержку (RTT).

Последствия решения

Положительные

- Снижение рисков: Постепенный подход, начиная с наименее критичного, но наиболее ресурсоемкого сервиса позволяет откатить любой выделенный сервис без остановки всей платформы.
- Изоляция ресурсов: Проблемы в аналитике или статистике (тяжёлые SQL) больше не влияют на критический RTB-путь.
- Новые интеграции (DSP) могут подключаться через фасад, используя очереди, не дожидаясь полной очистки легаси-кода.
- Горизонтальная масштабируемость: Возможность добавлять вычислительные мощности путем простого увеличения количества реплик и шардирования БД.
- Устранение единой точки отказа: Выход из строя сервиса статистики больше не влияет на способность системы отвечать на ставки.
- Ускорение разработки: команды разработки могут владеть отдельными сервисами целиком,

Отрицательные / Риски

- Сложность экспликации решения в переходный период: команде придется поддерживать как микросервисы так и монолит одновременно.
- Существует риск рассинхронизации данных между старой схемой монолита выделенными БД для микросервисов и новыми сервисами.
- Повышение сетевой задержки (Network Latency): Внутрисервисное взаимодействие (gRPC/HTTP) добавляет 1–3 мс.

- Усложняется end-to-end тестирование сквозных процессов (например, от клика до списания средств). Трассировка запроса, проходящего через 4–5 сервисов, требует внедрения распределённой трассировки